MINI PROJECT REPORT

OVERFLOW DETECTIVE: SIGNED NUMBER REPRESENTATION AND ARITHMETIC OVERFLOW DETECTION

Submitted by:

Easala Jagadeesh

Roll No.: 2411021061183

Email ID: easala.jagadeesh.s71@kalvium.community, jeasalaBTECH24@ced.alliance.edu.in

Squad: 71

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Under the guidance of

Moufia Justin

Associate Professor, ASAC

Alliance School of Advanced Computing

Alliance University, Bengaluru

October-2026



COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on **Computer Organization and Architecture** is submitted by **Easala Jagadeesh** [Uni. Reg. No. 2411021061183] in partial fulfilment for the award of the degree of Bachelor of Technology (Computer Science and Engineering) of Alliance University, is a bonafide work accomplished under my supervision and guidance during the academic year 2026-27. This report embodies the results of original work and studies conducted by the student and the contents do not form the basis for the award of any other degree to the candidate or anybody else.

- - - - - - - - - - - - - - - - - - - -

Moufia Justin

Associate Professor



COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled **OVERFLOW DETECTIVE: SIGNED NUMBER REPRESENTATION AND ARITHMETIC OVERFLOW DETECTION**, submitted by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (CSE) at Alliance University, is a record of my original work carried out under the supervision of Moufia Justin, Associate Professor.

I further confirm that all views, findings, and conclusions expressed herein are the result of sincere effort and academic inquiry, and have been reviewed and deliberated in consultation with the assigned mentor.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

Easala Jagadeesh	**2411021061183**	
Name of the Student	University Registration Number	Signature

Acknowledgement

I would like to express my heartfelt gratitude to everyone who contributed to the successful completion of this project titled **OVERFLOW DETECTIVE: SIGNED NUMBER REPRESENTATION AND ARITHMETIC OVERFLOW DETECTION**.

First and foremost, I extend my sincere thanks to my mentor and guide, Moufia Justin, for the unwavering support, insightful feedback, and encouragement throughout the development of this system. The guidance received in the subject Computer Organization and Architecture provided the perfect foundation for this work.

I am deeply grateful to my faculty members and peers whose guidance and technical inputs helped refine the project. Their collaborative spirit and constructive suggestions were instrumental in shaping the final outcome.

This project allowed me to look inside the arithmetic unit of a processor, to implement the number representations studied in class from first principles, and to understand why a few bits of register width have caused failures in real systems.

Lastly, I would like to thank everyone who supported me directly or indirectly during this journey. Your encouragement and belief in my work made this accomplishment possible.

Thank you.

Easala Jagadeesh

TABLE OF CONTENTS

Acknowledgement **4**

Abstract **6**

Chapter I- Introduction **7**

Problem Statement 7

Theory 7

Chapter II - System Configuration **9**

Algorithm 9

Implementation 10

Source Code 10

Chapter III - Results **14**

Sample inputs and outputs 14

Number conversion and representations 14

Bit-level addition 15

Real overflow failures 16

Verification 17

Chapter IV - Test Cases **18**

Number representations 18

Addition and overflow detection 18

Invalid input 19

Analysis of the test results 19

Chapter V - Conclusion **21**

Abstract

Processors store integers in registers of a fixed width, such as 8, 16 or 32 bits. When the result of an addition does not fit, the hardware does not stop: the result silently wraps around, usually to a number with the opposite sign. This integer overflow has caused real failures, including the YouTube view counter in 2014, a Boeing 787 generator-control counter in 2015 and the Ariane 5 rocket in 1996.

Overflow Detective is a simulator that converts numbers between positional systems, encodes signed integers in sign-magnitude, 1's complement and 2's complement, and adds two numbers bit by bit through a ripple-carry adder exactly as an arithmetic logic unit (ALU) does. It detects overflow with two rules: the sign rule (two operands of the same sign give a result of the other sign) and the hardware rule $V = C_{in}$ XOR C_{out} on the sign bit.

The simulator reproduces all four real-world failures. The project's own test suite passed all 256 four-bit cases and 8 boundary cases, and an additional exhaustive test of all 65,536 pairs of 8-bit numbers confirmed that both overflow rules always agree and that every non-overflowing result is exact.

The program takes the numbers to convert or add and the word size as input, and prints every bit, the carry chain and the overflow verdict. 17 of 17 test cases passed, covering conversions, the three representations, additions with and without overflow, and invalid input.

Chapter I- Introduction

Computer organisation describes how the parts of a computer are built and connected, and computer arithmetic is one of its foundations: the processor's arithmetic logic unit (ALU) works on fixed-width binary words, not on the unbounded integers of mathematics. Understanding how numbers are represented in those words, and what happens when a result does not fit, is essential for writing correct software.

This project builds a simulator that shows each step: converting numbers to binary, encoding negative numbers, adding bit by bit with carries, and raising the overflow flag, and applies it to well-known overflow failures.

Problem Statement

Integer overflow is silent: a 32-bit counter at 2,147,483,647 that is increased by one becomes −2,147,483,648 and the program continues with a wrong value. The problems addressed are:

- to convert integers between decimal, binary, octal and hexadecimal;
- to encode signed integers in the three standard representations and compare their ranges;
- to simulate fixed-width addition bit by bit and detect overflow with both the sign rule and the hardware carry rule; and
- to reproduce real overflow failures and determine the register width that would have prevented them.

Theory

Positional number systems

In base r a number is written as digits $d_m \ldots d_1\ d_0$ with value $\Sigma\ d_i \cdot r^i$. A decimal integer is converted to base r by repeated division: the remainders, read from last to first, are the digits. Octal and hexadecimal are compact ways of writing binary, because each octal digit stands for 3 bits and each hexadecimal digit for 4.

Signed number representations

Representation	Negative numbers	Range with n bits	Zero
Sign-magnitude	sign bit 1, then the magnitude	$-(2^{n-1}-1)$ to $2^{n-1}-1$	+0 and −0
1's complement	invert every bit of the positive value	$-(2^{n-1}-1)$ to $2^{n-1}-1$	+0 and −0
2's complement	invert every bit, then add 1	-2^{n-1} to $2^{n-1}-1$	one zero

Table 1: The three signed representations

Two's complement is used by practically all modern processors because it has a single zero, one more negative value, and, most importantly, the same adder circuit works for signed and unsigned numbers: subtraction is addition of the two's complement.

The ripple-carry adder

An n-bit adder is a chain of n full adders. The full adder for bit i adds A_i, B_i and the incoming carry C_i:

$$\mathbf{S_i = A_i \text{ XOR } B_i \text{ XOR } C_i,\ C_{i+1} = A_iB_i + C_i(A_i \text{ XOR } B_i)}$$

The carry ripples from the least significant bit to the most significant (sign) bit, and the carry out of the sign bit is discarded.

Detecting overflow

Rule	Overflow occurs when
Sign rule	both operands have the same sign bit and the result's sign bit is different
Hardware rule	$V = C_{in}$ XOR C_{out}: the carry into the sign bit differs from the carry out of it

Table 2: The two overflow rules

The two rules are equivalent. If both sign bits are 0, the sum's sign bit is just the carry into it, and no carry leaves it; overflow (result sign 1) happens exactly when $C_{in} = 1$ and $C_{out} = 0$. If both are 1, a carry always leaves the sign bit, and the result sign is 0 exactly when $C_{in} = 0$. If the signs differ, the sum always fits, and $C_{in} = C_{out}$. Processors implement the hardware rule as a single XOR gate that sets the overflow flag (V in ARM, OF in x86).

Chapter II - System Configuration

Overflow Detective is written in Python 3 without external libraries. It has a terminal program with a demonstration mode, an interactive mode and a built-in test suite, and a web interface with a converter, a bit-level adder and the case studies.

Component	Specification
Laptop	Dell Inspiron 15 3525
Processor	AMD Ryzen 5 5625U with Radeon Graphics, 6 cores / 12 threads
Memory	8 GB RAM
Operating system	Windows 11 Home Single Language
Language	Python 3.12 (standard library only, no external libraries)
Web interface	HTML5, CSS3, JavaScript (ES6), served by Python's built-in http.server
Browser	Microsoft Edge (Chromium)
Version control	Git and GitHub

Table 3: System configuration

Algorithm

Conversion by repeated division

1. While n > 0: append the remainder n mod r as the next digit and set n = n div r.
2. Read the digits in reverse order.

Two's complement encoding

1. Check that n lies in the range -2^{n-1} to $2^{n-1}-1$; otherwise it does not fit.
2. For $n \geq 0$ write n in binary, padded to the word size.
3. For $n < 0$ write $|n|$ in binary, invert every bit, add 1 and keep the lowest bits.

Bit-level addition with overflow detection

1. Encode both operands in two's complement.
2. For each bit from the least significant to the sign bit: total = $A_i + B_i$ + carry; the sum bit is total mod 2 and the new carry is total div 2. Record the carry into each bit.
3. Overflow (sign rule): the operands' sign bits are equal and the result's sign bit differs.
4. Overflow (hardware rule): V = carry into the sign bit XOR carry out of it; the program asserts that the two rules agree.
5. Decode the result bits back to a signed value and report the true sum for comparison.

Implementation

The project is written in Python 3 with the standard library only. Table 4 lists its files; the complete source code is included in the submitted ZIP file.

File	What it does
main.py	conversions, the three signed representations, the bit-level adder with overflow detection, the case studies, and the command line (--add, --convert, --bits, the demonstration, the interactive mode and the built-in test suite)
app.py, index.html	web interface: converter, bit-by-bit adder and case studies
api/index.py	the same functions as an online service for the deployed web app
tests/test_overflow.py	10 automated unit tests (Chapter IV)

Table 4: Files of the project

Inputs and outputs

Input: one or two integers and a word size of 4, 8, 16, 32 or 64 bits, for example `python main.py --add 100 50 --bits 8` or `python main.py --convert -5`; the interactive mode asks for the same values. **Output:** for --convert, the number in binary, octal and hexadecimal and in the three signed representations, its word range and whether it fits; for --add, the carries, both operands and the result bit by bit, the carry into and out of the sign bit, V, and the true sum for comparison. Operands that do not fit the word size, unsupported word sizes and non-numbers are rejected with a clear message.

Making the concept visible

The addition trace shows exactly what an ALU does: the carry entering every bit, the result bits, and the two carries at the sign bit whose XOR is the overflow flag V. The conversion output places the three representations side by side so that their difference is visible bit by bit. The web interface shows the same steps with the sign bit highlighted (Figures 1 and 2).

Source Code

The listings below are taken directly from the project file main.py.

Number conversion and two's complement

```
def to_base(n, base):
    """
    Convert a non-negative integer n to a string in the specified base (2, 8, or 16)
    using the repeated division method.
    """
    if n < 0:
        raise ValueError("to_base requires a non-negative integer.")
    if n == 0:
        return "0"

    digits = "0123456789ABCDEF"
    result = []
    current = n
    while current > 0:
```

```
        remainder = current % base
        result.append(digits[remainder])
        current //= base
    return "".join(reversed(result))

def twos_complement(n, bits):
    """
    Generate the 2's complement representation of an integer n in 'bits' width.
    - Positive numbers: standard binary representation padded to 'bits'.
    - Negative numbers: invert all bits of positive magnitude and add 1 (modulo 2^bits).
    Valid range: [-2^(bits-1), 2^(bits-1) - 1]
    """
    low, high = signed_range(bits)
    if not (low <= n <= high):
        return None  # Out of representable range

    if n >= 0:
        return to_base(n, 2).zfill(bits)

    # Invert positive magnitude and add 1
    pos_bits = to_base(-n, 2).zfill(bits)
    inverted = "".join("1" if b == "0" else "0" for b in pos_bits)
    numeric_sum = from_binary(inverted) + 1
    # Truncate to exact bit width
    return to_base(numeric_sum, 2).zfill(bits)[-bits:]

def read_twos_complement(bit_string):
    """
    Decode an n-bit two's complement binary string to its signed decimal value.
    If MSB is 1, value is negative: value = unsigned_value - 2^bits.
    """
    bits = len(bit_string)
    unsigned_val = from_binary(bit_string)
    if bit_string[0] == "1":
        return unsigned_val - (1 << bits)
    return unsigned_val
```

Bit-level ALU addition and overflow detection

```
def add_and_check(a, b, bits):
    """
    Simulate ALU bit-by-bit addition using a ripple full-adder model
    and evaluate arithmetic overflow.

    Overflow Conditions in Two's Complement:
    1. Sign-Consistency Rule:
       Overflow occurs if and only if two operands of the SAME sign yield
       a sum with the OPPOSITE sign.
    2. Hardware Carry-Bit Condition:
       Overflow V = Carry_in_to_MSB XOR Carry_out_from_MSB (V = C_in XOR C_out).
    """
    x = twos_complement(a, bits)
    y = twos_complement(b, bits)

    low, high = signed_range(bits)
    if x is None or y is None:
        raise ValueError(
            f"Operands must fit within {bits}-bit range [{low:,} to {high:,}]."
        )

    result_bits = ["0"] * bits
    carries = ["0"] * bits
    carry = 0

    # Ripple full adder from LSB (index bits - 1) to MSB (index 0)
    for i in range(bits - 1, -1, -1):
        carries[i] = str(carry)
        bit_a = int(x[i])
        bit_b = int(y[i])
        total = bit_a + bit_b + carry
        result_bits[i] = str(total % 2)
        carry = total // 2

    carry_in_msb = int(carries[0])
```

```
    carry_out_msb = carry
    result_str = "".join(result_bits)
    carries_str = "".join(carries)

    # Hardware overflow flag: V = Cin_msb XOR Cout_msb
    hardware_overflow = bool(carry_in_msb ^ carry_out_msb)
    # Sign rule: operands have same sign, but result sign differs
    sign_overflow = (x[0] == y[0]) and (result_str[0] != x[0])

    # Both conditions are mathematically identical in 2's complement
    assert hardware_overflow == sign_overflow

    cpu_answer = read_twos_complement(result_str)
    true_answer = a + b

    overflow_reason = ""
    if sign_overflow:
        if x[0] == "0":
            overflow_reason = (
                f"Positive overflow: operands {a} and {b} are positive, but result wrapped to negative {
    cpu_answer}."
            )
        else:
            overflow_reason = (
                f"Negative underflow/overflow: operands {a} and {b} are negative, but result wrapped to
    positive {cpu_answer}."
            )
    else:
        overflow_reason = f"Valid operation: mathematical sum {true_answer} fits within {bits}-bit
    range."

    return {
        "a_bits": x,
        "b_bits": y,
        "carries": carries_str,
        "carry_in_msb": carry_in_msb,
        "carry_out_msb": carry_out_msb,
        "result_bits": result_str,
        "cpu_answer": cpu_answer,
        "true_answer": true_answer,
        "overflow": sign_overflow,
        "overflow_reason": overflow_reason,
    }
```

The command-line inputs

```
def main():
    parser = argparse.ArgumentParser(
        description="Overflow Detective: Signed Number Representations & Overflow Analysis."
    )
    parser.add_argument(
        "-i", "--interactive", action="store_true", help="Launch interactive command-line interface."
    )
    parser.add_argument(
        "-t", "--test", action="store_true", help="Run internal validation test suite."
    )
    parser.add_argument(
        "-d", "--demo", action="store_true", help="Run academic demonstration."
    )
    parser.add_argument(
        "--add", nargs=2, type=int, metavar=("A", "B"),
        help="Add two integers with a bit-level ALU trace and overflow check, e.g. --add 100 50"
    )
    parser.add_argument(
        "--convert", type=int, metavar="N",
        help="Show N in binary, octal, hex, sign-magnitude, 1's and 2's complement, e.g. --convert -5"
    )
    parser.add_argument(
        "--bits", type=int, default=8, choices=(4, 8, 16, 32, 64), help="Word size for --add and --
    convert (default 8)"
    )

    args = parser.parse_args()
    if args.add or args.convert is not None:
```

```
    try:
        if args.convert is not None:
            display_conversion(args.convert, args.bits)
        if args.add:
            print(f"\n--- ALU Addition Trace: {args.add[0]} + {args.add[1]} ({args.bits}-bit) ---")
            display_addition(args.add[0], args.add[1], args.bits)
    except ValueError as err:
        print(f"Error: {err}")
        sys.exit(1)
elif args.test:
    run_unit_tests()
elif args.interactive:
    interactive_mode()
else:
    run_demonstration()
```

Chapter III - Results

Sample inputs and outputs

Two runs of the program are shown below exactly as it printed them: the addition 100 + 50 in 8 bits, and the conversion of −5.

Sample run 1: python main.py --add 100 50

```
--- ALU Addition Trace: 100 + 50 (8-bit) ---
  Configuration: 8-bit Two's Complement (Range: -128 to 127)
  Carries:      11000000
  Operand A:    01100100  (        100)
  Operand B:  + 00110010  (         50)
  ---------------------------
  ALU Result:   10010110  (       -106)
  Expected Sum:         150
  Hardware:     Cin(MSB) = 1, Cout(MSB) = 0 -> V = 1
  STATUS:       [OVERFLOW DETECTED] - Positive overflow: operands 100 and 50 are positive, but result
    wrapped to negative -106.
```

Sample run 2: python main.py --convert -5

```
--- Conversion for -5 (8-bit) ---
Absolute Binary: 101
Absolute Octal:  5
Absolute Hex:    5
Sign-Magnitude:  10000101
1's Complement:  11111010
2's Complement:  11111011
Word Range:      [-128 to 127]
Status:          Value fits within 8 bits.
```

In run 1 the true sum 150 is larger than 127, the largest 8-bit value. The carry row shows a carry entering the sign bit ($C_{in} = 1$) but none leaving it ($C_{out} = 0$), so V = 1 and the result bits 10010110 are read as −106. In run 2 the three representations of −5 differ only in how the negative value is formed: sign-magnitude sets the sign bit, 1's complement inverts every bit of 00000101, and 2's complement adds 1 to that.

Number conversion and representations

Decimal	Binary	Octal	Hexadecimal
13	1101	15	D
200	11001000	310	C8
2026	11111101010	3752	7EA

Table 5: Conversions by repeated division

Value	Sign-magnitude	1's complement	2's complement
5	00000101	00000101	00000101
-5	10000101	11111010	11111011
-1	10000001	11111110	11111111
-128	does not fit	does not fit	10000000

Table 6: The three representations in 8 bits

The value −128 fits only in two's complement, which has one more negative value because it has a single zero (Table 7).

Word size	Sign-magnitude / 1's complement	2's complement
4 bits	-7 to 7 (two zeros)	-8 to 7
8 bits	-127 to 127 (two zeros)	-128 to 127
16 bits	-32,767 to 32,767 (two zeros)	-32,768 to 32,767
32 bits	-2,147,483,647 to 2,147,483,647 (two zeros)	-2,147,483,648 to 2,147,483,647

Table 7: Ranges of the representations

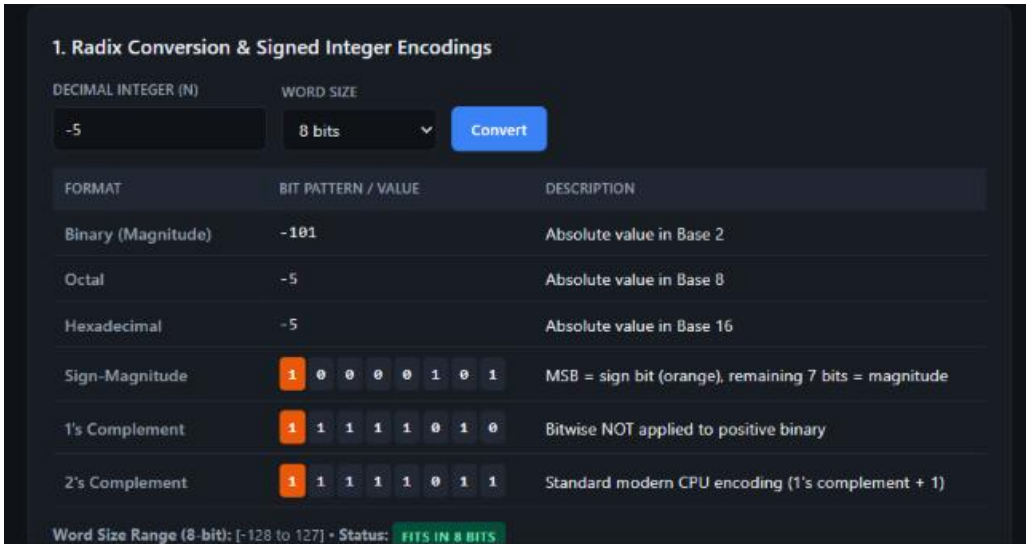


Figure 1: The converter showing −5 in the three representations; the sign bit is highlighted

Bit-level addition

Operation	A	B	Carries	Result	ALU result	C_{in}/C_{out}	Overflow
50 + 20	`00110010`	`00010100`	`01100000`	`01000110`	70	0, 0	no
100 + 50	`01100100`	`00110010`	`11000000`	`10010110`	-106	1, 0	yes
-100 + -50	`10011100`	`11001110`	`00111000`	`01101010`	106	0, 1	yes

Table 8: Three additions in 8-bit two's complement, traced by the simulator

50 + 20 = 70 fits. 100 + 50 should be 150, but the largest 8-bit value is 127: the carry enters the sign bit (C_{in} = 1) but none leaves it (C_{out} = 0), so V = 1 and the result wraps to −106. In the same way −100 + (−50) wraps to +106 with C_{in} = 0 and C_{out} = 1.



Figure 2: The bit-by-bit adder for 100 + 50 in 8 bits, with the carry chain and the overflow verdict

Real overflow failures

Case	Operation	Word size	ALU result	Fix: minimum word size
YouTube View Counter (2014)	2,147,483,647 + 1	32 bits	-2,147,483,648	64 bits
Boeing 787 Generator Control Unit (2015)	2,147,483,647 + 1	32 bits	-2,147,483,648	64 bits
Ariane 5 Flight 501 Disaster (1996)	32,767 + 1	16 bits	-32,768	32 bits
Classic Arcade / 8-Bit Score Counter	127 + 1	8 bits	-128	16 bits

Table 9: Real-world overflow cases reproduced by the simulator

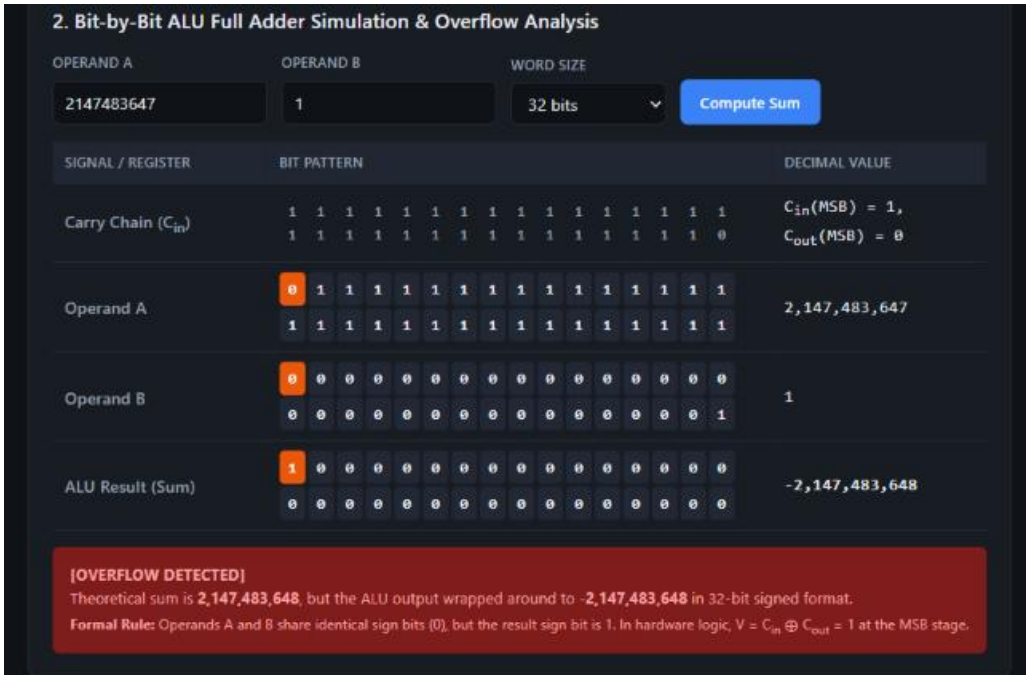


Figure 3: The YouTube view counter case: 2,147,483,647 + 1 in 32 bits wraps to −2,147,483,648

Verification

The project's built-in test suite reported: 'Passed all 256 4-bit exhaustive combinations and 8 boundary cases.'. In addition, all 65,536 pairs of 8-bit numbers (−128 to 127) were added: the sign rule and the hardware rule agreed in every case, 16,384 pairs overflowed exactly when the true sum was outside the range, and every other result equalled the true sum. Encoding and decoding every 16-bit value also returned the original number.

Chapter IV - Test Cases

Overflow Detective was tested on 17 test cases covering the number representations, additions with and without overflow, and invalid input. Every 'Actual output' below was produced by running the project code while this report was generated. The unit tests in tests/test_overflow.py (run with `python -m unittest discover tests`) check the same behaviour automatically.

Number representations

No.	Test case	Input	Expected output	Actual output	Result
1	Conversion of 13	--convert 13: binary, octal, hex	1101, 15, D	1101, 15, D	PASS
2	Conversion of 2026	--convert 2026: binary, octal, hex	11111101010, 3752, 7EA	11111101010, 3752, 7EA	PASS
3	Three forms of −5	--convert -5 --bits 8	SM 10000101; 1's 11111010; 2's 11111011	SM 10000101; 1's 11111010; 2's 11111011	PASS
4	−128 in 8 bits	--convert -128 --bits 8	SM: does not fit; 1's: does not fit; 2's: 10000000	SM: does not fit; 1's: does not fit; 2's: 10000000	PASS
5	Encode and decode	every 16-bit value −32,768 to 32,767	65,536 / 65,536 decode to the same number	65,536 / 65,536 decode to the same number	PASS
6	Word size needed	127, 128 and 2,147,483,648	8, 16 and 64 bits	8, 16 and 64 bits	PASS

Table 10: Test cases for the representations

Addition and overflow detection

No.	Test case	Input	Expected output	Actual output	Result
7	No overflow	--add 50 20 --bits 8	70; Cin 0, Cout 0, V = 0	70; Cin 0, Cout 0, V = 0	PASS
8	Positive overflow	--add 100 50 --bits 8	-106; Cin 1, Cout 0, V = 1	-106; Cin 1, Cout 0, V = 1	PASS
9	Negative overflow	--add -100 -50 --bits 8	106; Cin 0, Cout 1, V = 1	106; Cin 0, Cout 1, V = 1	PASS
10	Largest value + 1	--add 127 1 --bits 8	-128; Cin 1, Cout 0, V = 1	-128; Cin 1, Cout 0, V = 1	PASS
11	Carry out but no overflow	--add -1 1 --bits 8	0; Cin 1, Cout 1, V = 0	0; Cin 1, Cout 1, V = 0	PASS

No.	Test case	Input	Expected output	Actual output	Result
12	YouTube counter (32-bit)	--add 2147483647 1 --bits 32	-2,147,483,648; Cin 1, Cout 0, V = 1	-2,147,483,648; Cin 1, Cout 0, V = 1	PASS
13	Every 8-bit pair	all 256 × 256 pairs of 8-bit operands	V = sign rule = out of range in 65,536 / 65,536	V = sign rule = out of range in 65,536 / 65,536	PASS

Table 11: Test cases for addition and overflow

Invalid input

No.	Test case	Input	Expected output	Actual output	Result
14	Operand too big	python main.py --add 200 1	rejected: Operands must fit within 8-bit range [-128 to 127].	rejected: Operands must fit within 8-bit range [-128 to 127].	PASS
15	Unsupported word size	python main.py --add 1 1 --bits 12	rejected: argument --bits: invalid choice: '12' (choose from 4, 8, 16, 32, 64)	rejected: argument --bits: invalid choice: '12' (choose from 4, 8, 16, 32, 64)	PASS
16	Not a number	python main.py --add 1 x	rejected: argument --add: invalid int value: 'x'	rejected: argument --add: invalid int value: 'x'	PASS
17	Value too big for the word	python main.py --convert 300 --bits 8	Value exceeds 8 bits. Minimum required word size: 16 bits.	Value exceeds 8 bits. Minimum required word size: 16 bits.	PASS

Table 12: Input checking test cases

Analysis of the test results

17 of 17 test cases passed. The results confirm the theory:

- **The representations follow their definitions.** Repeated division gives the textbook binary, octal and hexadecimal digits; −5 differs in the three forms exactly as the rules say; and −128 fits only in two's complement, because it is the only form with a single zero.
- **The overflow flag is exactly V = C_{in} XOR C_{out}.** Overflow appeared only when two operands of the same sign gave a result of the other sign (100 + 50 → −106, −100 + (−50) → 106, 127 + 1 → −128). For −1 + 1 a carry leaves the sign bit, yet V = 0 because a carry also enters it: a carry out is not the same as overflow.
- **No exception in the whole 8-bit space.** For all 65,536 operand pairs the hardware rule, the sign rule and the range test gave the same verdict (65,536 / 65,536).

- **The real failures are reproduced at their true word size.** 2,147,483,647 + 1 in 32 bits wraps to −2,147,483,648, the YouTube counter case.
- **Invalid input is caught.** Operands that do not fit, unsupported word sizes and non-numbers are rejected with a clear message, and a value too large for the word is reported together with the word size it needs.

Chapter V - Conclusion

This report built a simulator of the integer arithmetic performed by a processor. Positional conversion, the three signed representations and a bit-level ripple-carry adder were implemented from first principles, and the comparison showed why two's complement is the standard: a single zero, a larger range and one adder for both signed and unsigned numbers.

Overflow was detected with both the sign rule and the hardware rule $V = C_{in}$ XOR C_{out}, and an exhaustive test over all 65,536 pairs of 8-bit operands confirmed that the two rules are equivalent. The simulator reproduced four real failures and showed the register width that would have prevented each one.

The program accepts the numbers and the word size as input and shows every bit of the work. All 17 of 17 test cases passed, from the three representations of −5 to the 32-bit YouTube counter and invalid input.

Future work includes subtraction, multiplication and division with their own overflow conditions, unsigned overflow and the carry flag, saturating arithmetic as used in signal processing, and a comparison with carry-lookahead adders.